QCrypt - Classical & Quantum Cryptography Engine | OOP C++ Project

Name: M.AbuBakar

Roll No: BSCS25109

OOP PROJECT QCrypt

Classical & Quantum Cryptography Engine

Object Oriented Programming

Inheritance & Polymorphism

Implemented Ciphers:

- Caesar
- ROT13
- Vigenere
- XOR
- RSA
- BB84 Quantum

Project Overview

QCrypt is a comprehensive cryptography engine built in C++ using Object-Oriented Programming. It implements five classical cipher algorithms and one quantum key distribution protocol (BB84), all unified under a single abstract base class hierarchy.

The project now includes a **Graphical User Interface (GUI)** built using Raylib. It features a WhatsApp-style messenger where users can chat with contacts (Hafiz Abdullah, Omer Abdullah, Tayyab Ahmed) and apply any of the implemented ciphers to encrypt their messages in real-time. Chat histories are persistently saved to text files.

The system encrypts and decrypts user-provided text using six different algorithms -- Caesar, ROT13, Vigenere, XOR, RSA, and BB84 Quantum -- demonstrating inheritance and runtime polymorphism throughout.

All standard C++ containers (`std::string`, `std::vector`) were strictly avoided. Custom dynamic containers (`MyString`, `MyVector<T>`) were built from scratch to handle dynamic memory allocation.

Functional Requirements

- Accept plaintext input from the user via an interactive Raylib GUI.
- Encrypt the input using six cipher algorithms (Caesar, ROT13, Vigenere, XOR, RSA, BB84).
- Decrypt ciphertext back to the original message for display.
- Save and load chat histories using file I/O operations (`chat_hafiz.txt`, etc.).
- Simulate BB84 Quantum Key Distribution with eavesdrop detection.
- Manage all ciphers through a unified polymorphic interface (`CryptoManager`).
- Implement dynamic arrays and strings from scratch using pointers and dynamic memory.

OOP Concepts Implemented

2.1 Inheritance

Inheritance Type	Example
Single Inheritance	`CaesarCipher` inherits from `SubstitutionCipher`
Multi-level Inheritance	`BB84Cipher` -> `QuantumCipher` -> `Cipher`
Hierarchical Inheritance	`CaesarCipher` and `ROT13Cipher` both inherit from `SubstitutionCipher`
Abstract Base Classes	`Cipher`, `MathEngine`, `KeyGenerator`, `Basis`, `QuantumChannel`, `Party`

2.2 Polymorphism

Type	Description
Pure Virtual Functions	`encrypt()`, `decrypt()`, `showInfo()`, `getName()`, `compute()`, `measure()`, `transmit()`, `prepare()`, `generate()`

Type	Description
Runtime Polymorphism	`CryptoManager` calls `encrypt()` on `Cipher*` without knowing derived type. The `MessengerGUI` dynamically switches ciphers based on user selection.
Dynamic Dispatch	`channel->transmit()` resolves to `IdealChannel`, `NoisyChannel`, or `EavesdroppedChannel` at runtime.
Virtual Destructors	`virtual ~Cipher()` ensures correct cleanup through base pointers.

2.3 Encapsulation

- All internal cipher keys, GUI state, and contact data are encapsulated as `private`.
- Proper Destructors (`virtual ~Cipher()`, `~MyVector()`, `~MyString()`) ensure correct memory cleanup, avoiding memory leaks when the GUI closes.

Complete Class Hierarchy

```
Cipher                          (abstract base - ALL ciphers)
|
+-- SubstitutionCipher          (abstract - character shifting)
|   +-- CaesarCipher            shifts each letter by a fixed number
|   +-- ROT13Cipher             fixed shift of 13, self-inverse
|
+-- PolyalphabeticCipher        (abstract - keyword-based shifting)
|   +-- VigenereCipher          different shift per character using keyword
|
+-- BitwiseCipher               (abstract - bit-level operations)
|   +-- XORCipher               XOR each byte with a key
|
+-- AsymmetricCipher            (abstract - public/private key system)
|   +-- RSACipher               prime numbers + modular arithmetic
|
+-- QuantumCipher               (abstract - quantum key distribution)
    +-- BB84Cipher              BB84 protocol + derived XOR encryption

MathEngine                      (abstract base - mathematical operations)
+-- ModularArithmetic           fast exponentiation, GCD, modular inverse
+-- PrimeEngine                 primality testing, prime generation

KeyGenerator                    (abstract base - key creation)
+-- PrimeKeyGenerator           generates RSA key pair (p, q, e, d)
```

```
+-- SimpleKeyGenerator              generates shift value / keyword

Basis                               (abstract base - quantum measurement)
+-- RectilinearBasis (+)            measures 0 deg (bit 0) and 90 deg (bit 1)
+-- DiagonalBasis (x)               measures 45 deg (bit 0) and 135 deg (bit 1)

QuantumChannel                      (abstract base - photon transmission)
+-- IdealChannel                    photon arrives unchanged, no Eve
+-- NoisyChannel                    10% random photon disturbance
+-- EavesdroppedChannel             Eve intercepts - causes 25% error rate

Party                               (abstract base - protocol participants)
+-- Alice                           sender - prepares and sends photons
+-- Bob                             receiver - measures incoming photons
+-- Eve                             eavesdropper - intercepts in channel

Custom Utilities
+-- MyString                        Dynamic string implementation
+-- MyVector&lt;T&gt;                     Dynamic resizable array template
+-- RandomGen                       Pseudo-random number generator

Core Engine
+-- CryptoManager                   manages all ciphers, runs them together

GUI &amp; App Components
+-- MessengerApp                    Console-based fallback app
+-- MessengerGUI                    Raylib graphical interface
+-- Contact                         Encapsulates contact information
+-- Message                         Encapsulates message text and metadata
```

Module Breakdown

Module 1 - Math Engine

Class	Responsibility
MathEngine	Abstract base with pure virtual `compute(a, b, mod)`
ModularArithmetic	Fast modular exponentiation, GCD (Euclidean), Modular Inverse (Extended Euclidean)
PrimeEngine	Trial division primality test, random prime generation

Module 2 - Key Generator

Class	Responsibility
KeyGenerator	Abstract base with `generate()` and `show()`
PrimeKeyGenerator	Generates RSA key pair: finds p, q, computes n, phi(n), e, d
SimpleKeyGenerator	Stores a shift number and/or keyword string

Module 3 - Classical Ciphers

Class	Algorithm
CaesarCipher	Shift each letter by key positions in alphabet
ROT13Cipher	Fixed shift of 13 - encoding and decoding use same function
VigenereCipher	Shift each letter by corresponding keyword letter
XORCipher	XOR each byte with the key byte
RSACipher	Encrypt: C = M^e mod n, Decrypt: M = C^d mod n

Module 4 - Quantum Layer (BB84)

Class	Responsibility
Basis	Abstract measurement framework
RectilinearBasis	Correctly measures 0/90 degree photons
DiagonalBasis	Correctly measures 45/135 degree photons
QuantumChannel	Abstract photon transmission medium
IdealChannel	Passes photon unchanged
NoisyChannel	10% random disturbance
EavesdroppedChannel	Eve randomly disturbs photons - creates detectable errors
Party	Abstract base for protocol participants
Alice	Prepares random bits and random bases, encodes as polarization
Bob	Picks random bases, measures received photons
Eve	Intercepts channel with her own random basis

Class	Responsibility
BB84Cipher	Runs full protocol, extracts shared key, uses it for XOR

Module 5 – Core Engine

Class	Responsibility
CryptoManager	Holds an array of `Cipher*` base pointers. Manages the polymorphic cipher array and delegates `encrypt()` and `decrypt()` calls to the chosen derived class.
RandomGen	Simple pseudo-random number generator for BB84 bits/bases and prime generation.

Module 6 – App & GUI

Class	Responsibility
MessengerGUI	Manages Raylib window, 60fps rendering loop, sidebar contact selection, input handling, and automatic chat scrolling.
MessengerApp	Console-based alternative to the GUI messenger.
Contact	Encapsulates data for a user's contact (name, phone, chat file).
Message	Encapsulates a sent/received message's plain text, encrypted text, and cipher.

Cipher Algorithms

5.1 Caesar Cipher

Shifts each letter forward by key positions in the alphabet.

Formula: `C = (M + key) mod 26|M = (C - key + 26) mod 26`

5.2 ROT13 Cipher

A special Caesar cipher with fixed shift of 13. Since 13+13=26, encoding equals decoding.

5.3 Vigenere Cipher

Each letter is shifted by the corresponding letter of a repeating keyword.

Caesar because each letter has a different shift -- frequency analysis is ineffective.

5.4 XOR Cipher

Each byte is XOR'd with the key byte. XOR is its own inverse: encrypt = decrypt.

5.5 RSA-Style Cipher

Uses real number theory. Mathematics are identical to actual RSA with smaller key sizes for educational purposes.

5.6 BB84 Quantum Cipher

BB84 is a Quantum Key Distribution (QKD) protocol. Its security is guaranteed by the laws of quantum physics: measuring a quantum state disturbs it.

Protocol Steps:

Alice sends photons with random bits and random bases.

Bob randomly picks measurement bases.

When bases match, Bob gets the correct bit. When bases differ, Bob gets a random result.

Basis reconciliation: Alice and Bob publicly compare which bases they used (not the bits).

Extract shared secret key from positions where bases matched.

Eavesdrop detection: Compare a sample of key bits. If error rate exceeds 20%, Eve is detected and transmission is aborted.

Use shared key for XOR encryption of the message.

Polymorphism Demonstration

CryptoManager::runAll() / getCipher() - The Core Polymorphic Loop

`ciphers[i]` is a `Cipher*` -- an abstract base class pointer. At runtime, it resolves to the correct derived class:

- Slot 0 -> `CaesarCipher::encrypt()`
- Slot 1 -> `ROT13Cipher::encrypt()`
- Slot 2 -> `VigenereCipher::encrypt()`
- Slot 3 -> `XORCipher::encrypt()`
- Slot 4 -> `RSACipher::encrypt()`
- Slot 5 -> `BB84Cipher::encrypt()`

The loop never changes. The behavior changes completely based on the derived type.